

PROJECT REPORT

Movie Recommendation System using NLP, Machine Learning, and Streamlit.

1. Project Overview

The **Movie Recommendation System** is an end-to-end machine learning application that recommends movies similar to a user's selected movie. The recommendation engine uses **Natural Language Processing (NLP)** techniques to analyze movie metadata and suggests relevant movies based on content similarity.

To make the system interactive and user-friendly, the recommendation model is deployed using **Streamlit**, allowing users to search for a movie and instantly receive personalized recommendations along with movie posters.

2. Problem Statement

With thousands of movies available across streaming platforms, users often struggle to find content that matches their interests. Traditional searching based on genres or popularity may not always provide relevant recommendations.

The objective of this project is to build a **content-based recommendation system** that analyzes movie information and recommends similar movies using machine learning techniques.

3. Objectives

- Build a Content-Based Movie Recommendation System.
 - Perform data cleaning and preprocessing.
 - Apply NLP techniques to textual movie data.
 - Generate movie feature vectors using TF-IDF.
 - Calculate movie similarity using Cosine Similarity.
 - Deploy the recommendation model using Streamlit.
 - Create an interactive web application with movie posters and recommendations.
-

4. Technologies Used

Technology	Purpose
Python	Programming Language
Pandas	Data Manipulation
NumPy	Numerical Computation
NLTK	Text Preprocessing
Scikit-learn	TF-IDF & Cosine Similarity
Pickle	Model Serialization
Streamlit	Web Application
TMDB API	Fetching Movie Posters
Jupyter Notebook	Model Development

5. Dataset

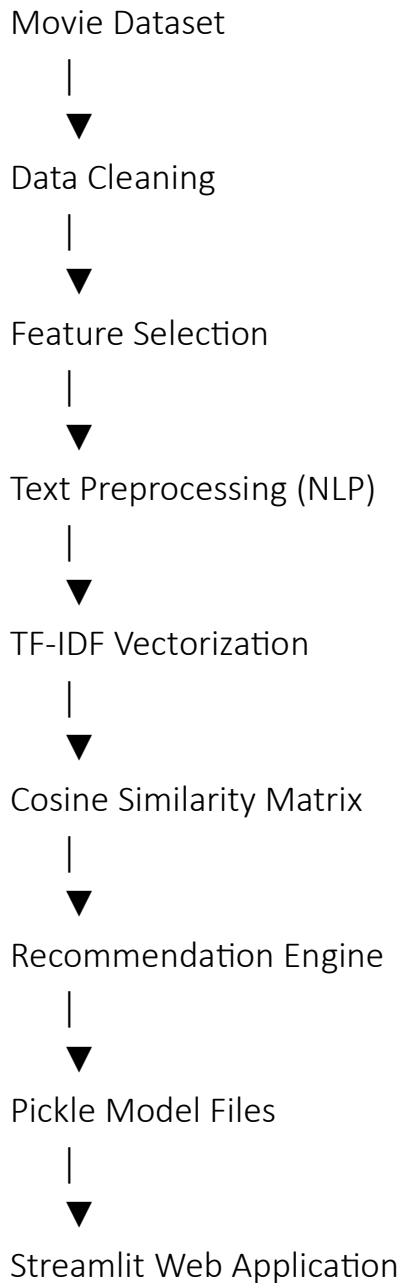
The project uses a movie dataset containing metadata for thousands of movies.

The following attributes are primarily used:

- Movie Title
- Overview
- Genres
- Tagline
- Popularity
- Average Rating

These features provide meaningful information about each movie and help the recommendation engine identify similar movies.

6. Project Workflow



7. Step-by-Step Implementation

Step 1: Import Libraries

Required Python libraries such as **Pandas**, **NumPy**, **NLTK**, **Scikit-learn**, **Pickle**, and **Streamlit** are imported.

These libraries are used for data manipulation, NLP preprocessing, machine learning, model serialization, and application deployment.

Step 2: Load the Dataset

The movie dataset is loaded into a Pandas DataFrame.

The dataset is inspected using functions such as:

- `head()`
- `shape`
- `info()`
- `describe()`

This helps understand the structure and quality of the data.

Step 3: Data Cleaning

The dataset undergoes several preprocessing steps:

- Remove duplicate records
- Handle missing values
- Select only the required columns
- Replace missing text with empty strings

Cleaning the dataset improves model performance and prevents processing errors.

Step 4: Feature Engineering

The **Genres** column is stored in JSON format.

Using Python's `ast.literal_eval()`, the genre information is converted into readable text.

A new feature called **Tags** is then created by combining:

- Overview
- Genres
- Tagline

This combined text becomes the primary feature for recommendation.

Step 5: Natural Language Processing (NLP)

Before training the recommendation model, the text is preprocessed.

The preprocessing steps include:

- Convert text to lowercase
- Remove punctuation and special characters
- Remove English stopwords
- Perform lemmatization

These steps reduce unnecessary words and improve the quality of feature extraction.

Step 6: TF-IDF Vectorization

Since machine learning algorithms cannot understand text directly, the processed text is converted into numerical vectors using **TF-IDF (Term Frequency–Inverse Document Frequency)**.

The vectorizer:

- Uses up to 50,000 important words
- Considers both unigrams and bigrams
- Ignores common English stopwords

This creates meaningful numerical representations of movie descriptions.

Step 7: Cosine Similarity

After vectorization, Cosine Similarity is used to measure how similar two movies are.

Similarity scores range between **0 and 1**.

Movies with higher similarity scores are considered more relevant and are recommended to the user.

Step 8: Recommendation Function

A custom recommendation function performs the following tasks:

1. Accepts a movie title from the user.
2. Finds the corresponding movie index.
3. Calculates similarity scores.
4. Sorts the movies based on similarity.

5. Returns the Top 10 recommended movies.

This forms the core logic of the recommendation engine.

Step 9: Model Serialization

To avoid retraining the model every time the application starts, the following objects are saved using Pickle:

- Processed movie dataset
- TF-IDF vectorizer
- Similarity matrix

This significantly improves application loading speed.

Step 10: Streamlit Web Application

A web application is developed using **Streamlit** to make the recommendation system interactive.

Features of the Application

- Simple and user-friendly interface
- Movie selection through a dropdown menu
- One-click recommendation generation
- Displays movie posters using the TMDB API
- Fast recommendation response using pre-trained models
- Responsive layout with a modern Netflix-inspired design

The Streamlit application transforms the machine learning model into a real-world, deployable recommendation system.

8. Results

The developed recommendation system successfully:

- Cleans and preprocesses movie data
- Extracts meaningful textual features

- Converts text into numerical vectors
- Computes movie similarity efficiently
- Recommends highly similar movies
- Displays recommendations through an interactive Streamlit application with movie posters

The application provides fast and relevant recommendations without requiring user ratings or watch history.

9. Advantages

- Easy to use
 - Fast recommendation generation
 - No user history required
 - Interactive web interface
 - Scalable architecture
 - Easily deployable on cloud platforms
-

10. Limitations

- Recommendations depend only on movie content
 - Does not learn from user behavior
 - Performance depends on the quality of movie metadata
 - New movies with limited descriptions may produce less accurate recommendations
-

11. Future Enhancements

The project can be further improved by:

- Implementing Collaborative Filtering
- Building a Hybrid Recommendation System
- Using Deep Learning embeddings such as BERT or Sentence Transformers
- Adding user authentication

- Maintaining watch history and favorite lists
 - Integrating movie trailers and reviews
 - Deploying the application on Streamlit Cloud or AWS
 - Providing personalized recommendations based on user preferences
-

12. Conclusion

This project demonstrates the successful development of a **Content-Based Movie Recommendation System** using **Natural Language Processing, Machine Learning, and Streamlit**. By combining data preprocessing, TF-IDF vectorization, and Cosine Similarity, the system effectively recommends movies with similar content. The integration of a Streamlit web application and TMDB API enhances usability by providing an interactive interface with movie posters, making the project a complete end-to-end recommendation solution suitable for real-world deployment.